Experiment 11: CNN for Image Classification

Question

Problem Statement: A recycling plant wants a camera system that sorts waste on a conveyor belt into categories automatically. A garbage-classification image dataset (cardboard, glass, metal, paper, plastic, trash), or CIFAR-10 as a substitute. The objective is to build and train a CNN for multi-class image classification. Use TensorFlow, Keras, NumPy, Matplotlib.

Tasks:

1. Load, resize and normalise the images; split into train, validation and test sets.
2. Build a CNN with convolution, pooling, flatten and dense layers.
3. Train with categorical cross-entropy and plot accuracy and loss curves.
4. Show the confusion matrix and sample misclassifications.
5. Visualise the feature maps of the first convolution layer.

Code

```
"""Experiment 11: CNN for Image Classification (CIFAR-10 used as the dataset)"""
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, classification_report

np.random.seed(42); tf.random.set_seed(42)
USE_SYNTHETIC = False        # keep False; True only generates random images to smoke-test the code
class_names = ["airplane", "automobile", "bird", "cat", "deer", "dog", "frog", "horse", "ship", "truck"]

# ---------------- Task 1: load, normalise, split ----------------
if USE_SYNTHETIC:
    x_all = np.random.randint(0, 256, (3000, 32, 32, 3), dtype=np.uint8); y_all = np.random.randint(0, 10,
(3000, 1))
    x_test = np.random.randint(0, 256, (600, 32, 32, 3), dtype=np.uint8); y_test = np.random.randint(0, 10,
(600, 1))
else:
    (x_all, y_all), (x_test, y_test) = keras.datasets.cifar10.load_data()

# For the garbage-classification folder dataset use this instead (images resized to 128x128):
# ds = keras.utils.image_dataset_from_directory("garbage_dataset/", image_size=(128, 128), batch_size=32)

x_all = x_all.astype("float32") / 255.0                    # normalise to 0-1
x_test = x_test.astype("float32") / 255.0
y_all, y_test = y_all.ravel(), y_test.ravel()
n_val = int(0.1 * len(x_all))
x_train, y_train = x_all[:-n_val], y_all[:-n_val]
x_val, y_val = x_all[-n_val:], y_all[-n_val:]
print("Train:", x_train.shape, "Val:", x_val.shape, "Test:", x_test.shape)

y_train_c = keras.utils.to_categorical(y_train, 10)
y_val_c = keras.utils.to_categorical(y_val, 10)
y_test_c = keras.utils.to_categorical(y_test, 10)

# ---------------- Task 2: build CNN ----------------
model = keras.Sequential([
    layers.Input((32, 32, 3)),
    layers.Conv2D(32, 3, activation="relu", padding="same"),
    layers.Conv2D(32, 3, activation="relu", padding="same"),
    layers.MaxPooling2D(),
    layers.Conv2D(64, 3, activation="relu", padding="same"),
    layers.Conv2D(64, 3, activation="relu", padding="same"),
    layers.MaxPooling2D(),
    layers.Flatten(),
    layers.Dense(128, activation="relu"),
    layers.Dropout(0.5),
    layers.Dense(10, activation="softmax"),
```

```
])
model.summary()


# ---------------- Task 3: train with categorical cross-entropy ----------------
model.compile(optimizer="adam", loss="categorical_crossentropy", metrics=["accuracy"])
hist = model.fit(x_train, y_train_c, validation_data=(x_val, y_val_c), epochs=10, batch_size=64, verbose=2)


fig, ax = plt.subplots(1, 2, figsize=(12, 4))
ax[0].plot(hist.history["accuracy"], label="train"); ax[0].plot(hist.history["val_accuracy"],
label="validation")
ax[0].set_title("Accuracy"); ax[0].set_xlabel("Epoch"); ax[0].legend()
ax[1].plot(hist.history["loss"], label="train"); ax[1].plot(hist.history["val_loss"], label="validation")
ax[1].set_title("Loss"); ax[1].set_xlabel("Epoch"); ax[1].legend()
plt.tight_layout(); plt.show()

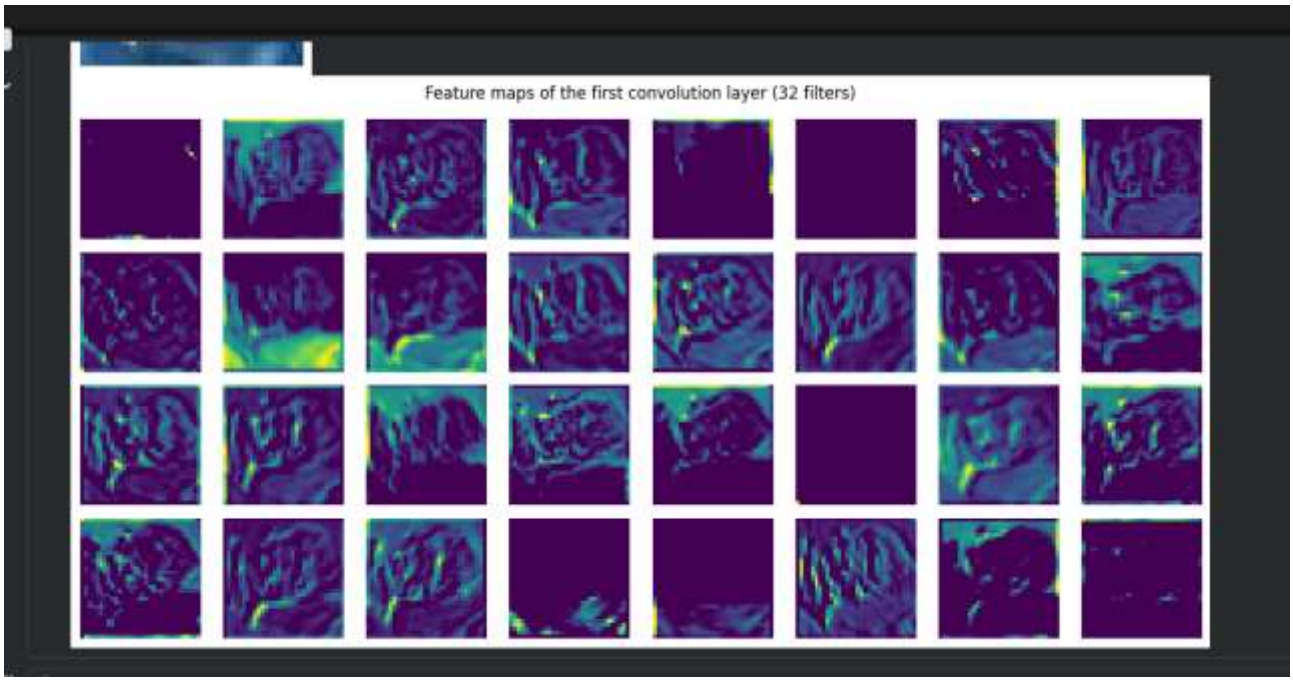

test_loss, test_acc = model.evaluate(x_test, y_test_c, verbose=0)
print(f"Test accuracy: {test_acc:.4f}")

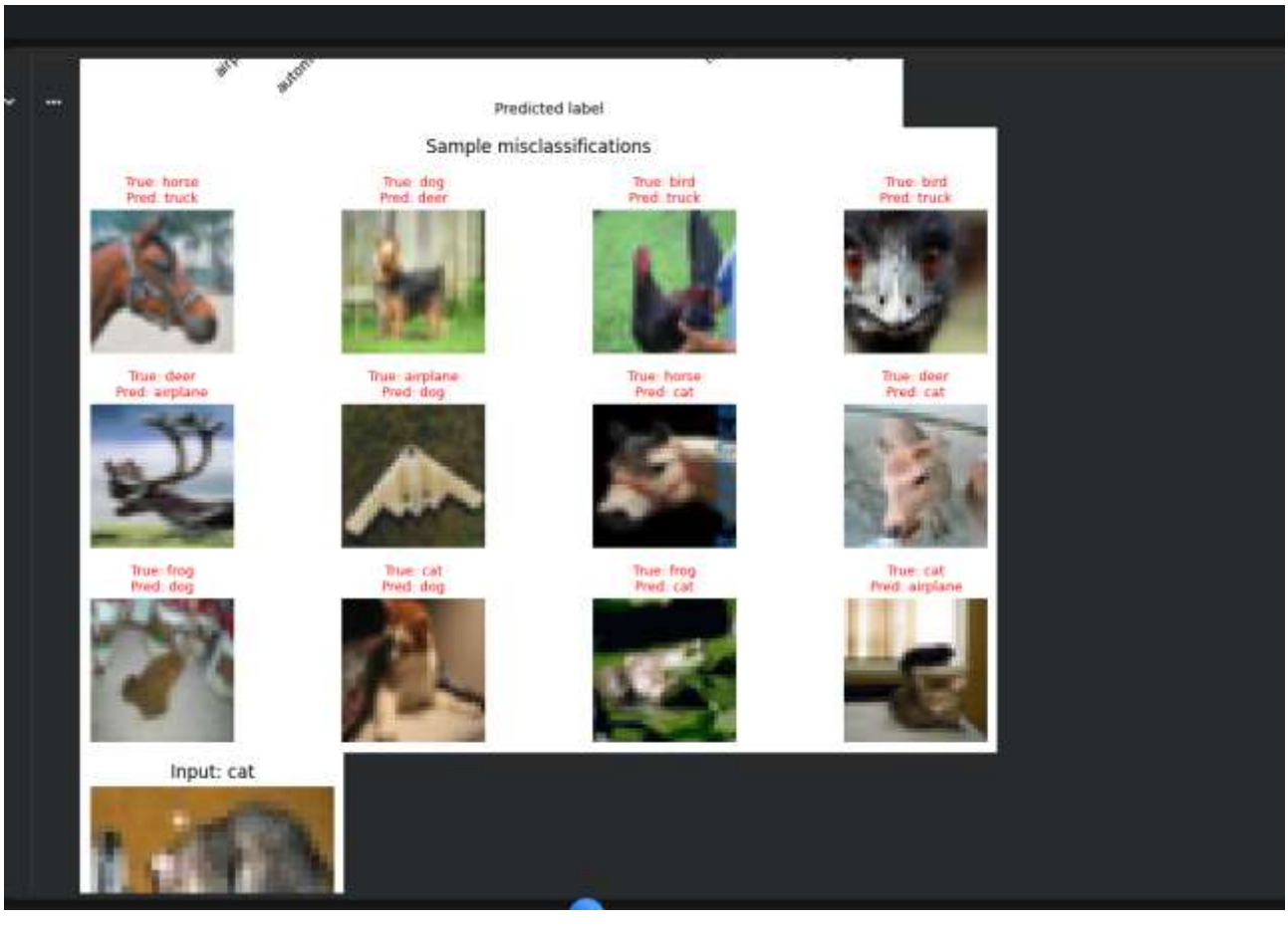

# ---------------- Task 4: confusion matrix & misclassifications ----------------
probs = model.predict(x_test, verbose=0)
y_pred = probs.argmax(axis=1)
print(classification_report(y_test, y_pred, target_names=class_names, zero_division=0))
cm = confusion_matrix(y_test, y_pred, labels=range(10))
fig, ax = plt.subplots(figsize=(8, 8))
ConfusionMatrixDisplay(cm, display_labels=class_names).plot(ax=ax, cmap="Blues", xticks_rotation=45,
colorbar=False)
plt.title("Confusion matrix"); plt.tight_layout(); plt.show()

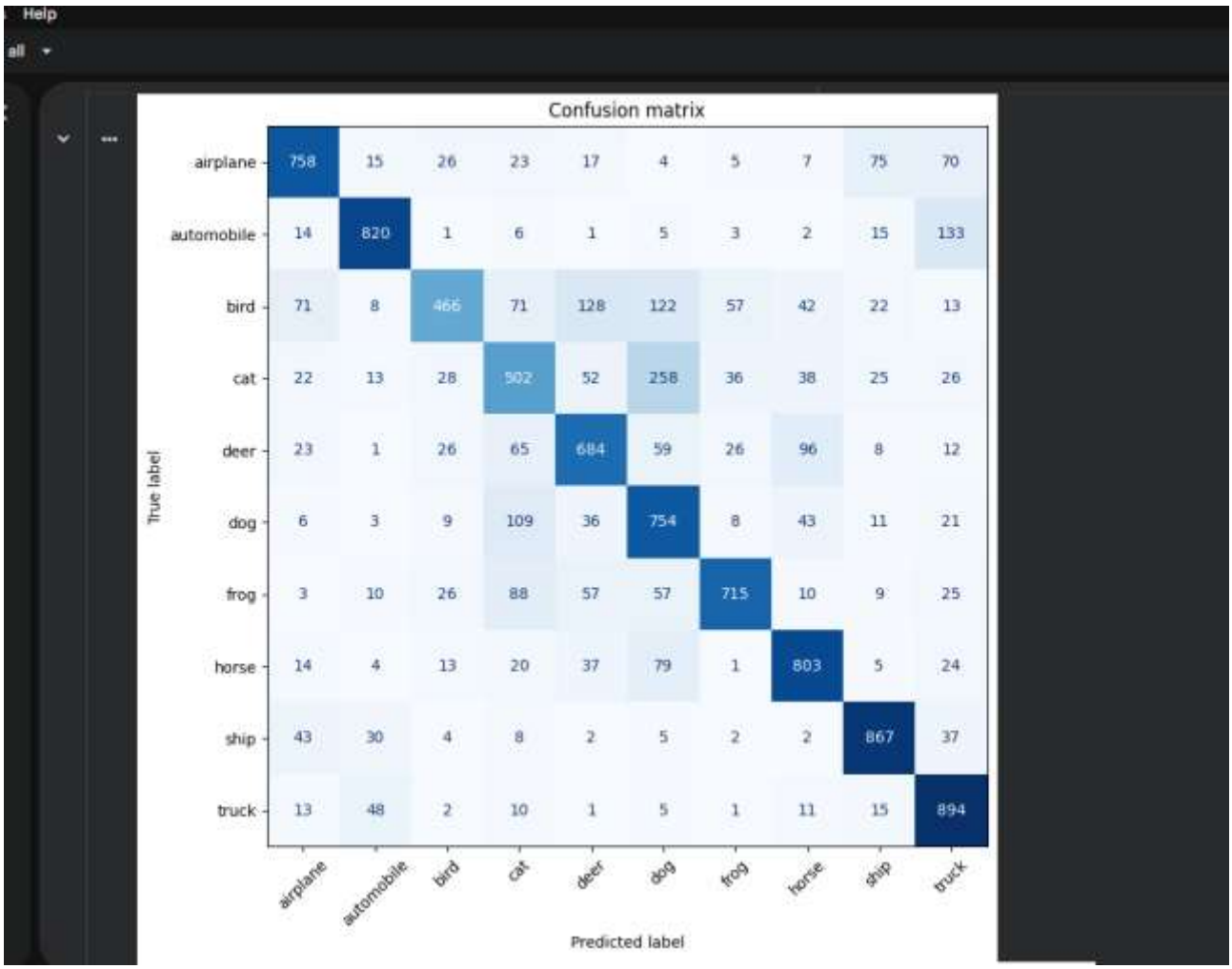

wrong = np.where(y_pred != y_test)[0][:12]
plt.figure(figsize=(10, 6))
for i, idx in enumerate(wrong):
    plt.subplot(3, 4, i + 1)
    plt.imshow(x_test[idx]); plt.axis("off")
    plt.title(f"True: {class_names[y_test[idx]]}\nPred: {class_names[y_pred[idx]]}", fontsize=9, color="red")
plt.suptitle("Sample misclassifications"); plt.tight_layout(); plt.show()

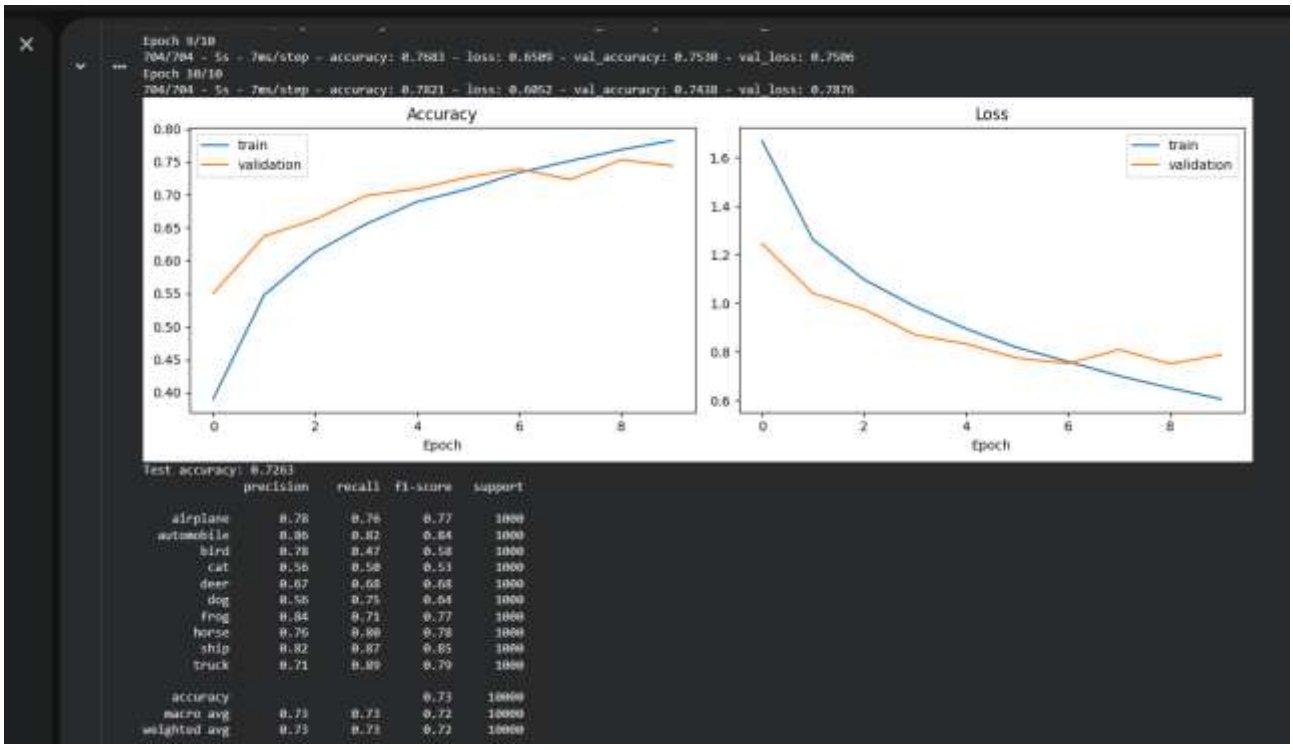

# ---------------- Task 5: feature maps of first conv layer ----------------
fm_model = keras.Model(inputs=model.inputs, outputs=model.layers[0].output)
sample = x_test[0:1]
fmaps = fm_model.predict(sample, verbose=0)[0]            # (32, 32, 32)
plt.figure(figsize=(3, 3)); plt.imshow(sample[0]); plt.title(f"Input: {class_names[y_test[0]]}");
plt.axis("off"); plt.show()
plt.figure(figsize=(12, 6))
for i in range(32):
    plt.subplot(4, 8, i + 1); plt.imshow(fmaps[:, :, i], cmap="viridis"); plt.axis("off")
plt.suptitle("Feature maps of the first convolution layer (32 filters)"); plt.tight_layout(); plt.show()
```

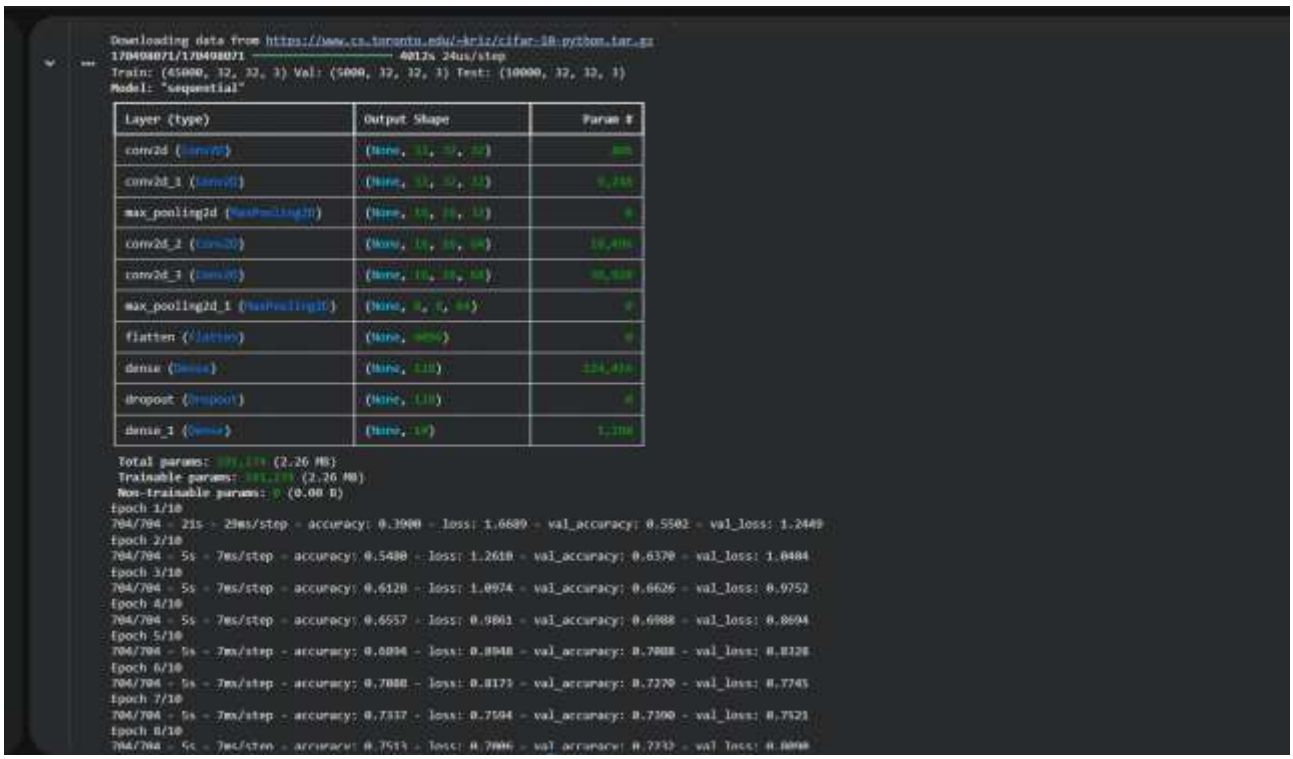
Output


Feature maps of the first convolution layer (32 filters)


Predicted label
Sample misclassifications
True: horse
Pred: truck
True: dog
Pred: deer
True: bird
Pred: truck
True: bird
Pred: truck
True: deer
Pred: airplane
True: airplane
Pred: dog
True: horse
Pred: cat
True: deer
Pred: cat
True: frog
Pred: dog
True: cat
Pred: dog
True: frog
Pred: cat
True: cat
Pred: airplane
Input: cat


Help
all
Confusion matrix
airplane 758 15 26 23 17 4 5 7 75 70
automobile 14 820 1 6 1 5 3 2 15 133
bird 71 8 466 71 128 122 57 42 22 13
cat 22 13 28 502 52 258 36 38 25 26
deer 23 1 26 65 684 59 26 96 8 12
dog 6 3 9 109 36 754 8 43 11 21
frog 3 10 26 88 57 57 715 10 9 25
horse 14 4 13 20 37 79 1 803 5 24
ship 43 30 4 8 2 5 2 2 867 37
truck 13 48 2 10 1 5 1 11 15 894
True label
airplane automobile bird cat deer dog frog horse ship truck
Predicted label


Epoch 9/10
704/704 - 5s - 7ms/step - accuracy: 0.7683 - loss: 0.6589 - val_accuracy: 0.7530 - val_loss: 0.7506
Epoch 10/10
704/704 - 5s - 7ms/step - accuracy: 0.7821 - loss: 0.6052 - val_accuracy: 0.7438 - val_loss: 0.7876
Accuracy
train
validation
Epoch
Loss
train
validation
Epoch
Test accuracy: 0.7263
precision recall f1-score support
airplane 0.78 0.76 0.77 1000
automobile 0.86 0.82 0.84 1000
bird 0.78 0.47 0.58 1000
cat 0.56 0.50 0.53 1000
deer 0.67 0.68 0.68 1000
dog 0.56 0.75 0.64 1000
frog 0.84 0.71 0.77 1000
horse 0.76 0.80 0.78 1000
ship 0.82 0.87 0.85 1000
truck 0.71 0.89 0.79 1000
accuracy 0.73 10000
macro avg 0.73 0.73 0.72 10000
weighted avg 0.73 0.73 0.72 10000


Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 ━━━━━━━━━━━━━━━━━━━━ 4012s 24us/step
Train: (45000, 32, 32, 3) Val: (5000, 32, 32, 3) Test: (10000, 32, 32, 3)
Model: "sequential"
Layer (type) Output Shape Param #
conv2d (Conv2D) (None, 32, 32, 32) 896
conv2d_1 (Conv2D) (None, 32, 32, 32) 9,248
max_pooling2d (MaxPooling2D) (None, 16, 16, 32) 0
conv2d_2 (Conv2D) (None, 16, 16, 64) 18,496
conv2d_3 (Conv2D) (None, 16, 16, 64) 36,928
max_pooling2d_1 (MaxPooling2D) (None, 8, 8, 64) 0
flatten (Flatten) (None, 4096) 0
dense (Dense) (None, 128) 524,416
dropout (Dropout) (None, 128) 0
dense_1 (Dense) (None, 10) 1,290
Total params: 591,274 (2.26 MB)
Trainable params: 591,274 (2.26 MB)
Non-trainable params: 0 (0.00 B)
Epoch 1/10
704/704 - 21s - 29ms/step - accuracy: 0.3900 - loss: 1.6689 - val_accuracy: 0.5502 - val_loss: 1.2449
Epoch 2/10
704/704 - 5s - 7ms/step - accuracy: 0.5480 - loss: 1.2618 - val_accuracy: 0.6370 - val_loss: 1.0404
Epoch 3/10
704/704 - 5s - 7ms/step - accuracy: 0.6128 - loss: 1.0974 - val_accuracy: 0.6626 - val_loss: 0.9752
Epoch 4/10
704/704 - 5s - 7ms/step - accuracy: 0.6557 - loss: 0.9861 - val_accuracy: 0.6988 - val_loss: 0.8694
Epoch 5/10
704/704 - 5s - 7ms/step - accuracy: 0.6894 - loss: 0.8948 - val_accuracy: 0.7088 - val_loss: 0.8326
Epoch 6/10
704/704 - 5s - 7ms/step - accuracy: 0.7088 - loss: 0.8373 - val_accuracy: 0.7270 - val_loss: 0.7745
Epoch 7/10
704/704 - 5s - 7ms/step - accuracy: 0.7337 - loss: 0.7594 - val_accuracy: 0.7390 - val_loss: 0.7521
Epoch 8/10
704/704 - 5s - 7ms/step - accuracy: 0.7513 - loss: 0.7006 - val_accuracy: 0.7732 - val_loss: 0.6868